AgPlane: Letting Agents Harness Their Own System-Level Behavior with eBPF

Paper #XXX
Anonymous Submission

Abstract

AI agents operate through harnesses that maintain tools and memory, and apply behavioral constraints for safe, effective operation. In practice, projects specify these constraints in natural language: our empirical study of 64 projects finds that 64% of instruction-file statements are behavioral directives, e.g. do not leak secrets, run tests before committing. Central to such a harness is a runtime policy engine that enforces these constraints deterministically over the agent's actions. Yet existing policy engines fail to connect agent intent to system-level actions, leaving a *semantic gap*. Tool-call guardrails can miss indirect system-level actions (83% of directives) and cross-event state (81% of projects); OS-level mechanisms see all actions but expect static policy, unable to resolve the 74% of rules that need intent level project or task context, and return opaque errors without semantic feedback. Our key observation is that the required context is mainly within the agent itself, so the engine should let agents dynamically define and adapt policy, and take effect at OS level for their own actions. We introduce AgPlane, a policy engine that allows agents to declare cross-event behavior as labeled information-flow control (IFC) policies under a layered authority model that ensures higher-authority rules cannot be weakened by agent-authored policy. AgPlane observes and enforces actions across process, file, and network boundaries with semantic feedback. We implement AgPlane with eBPF and evaluate it across empirically studied policies, system workloads, and coding tasks, showing improved policy compliance by 27pp with 2% end-to-end overhead on agent workloads.

CCS Concepts: • **Computer systems organization** → *Embedded and cyber-physical systems*; • **Security and privacy** → *Software and application security*; • **Software and its engineering** → *Software safety*.

Keywords: AI agents, eBPF, BPF-LSM, labeled information-flow control, information-flow control, provenance, semantic feedback

ACM Reference Format:
Paper #XXX. 2026. AgPlane: Letting Agents Harness Their Own System-Level Behavior with eBPF. In *Proceedings of 2026 ACM SIGOPS Annual Technical Conference (ATC '26).* ACM, New York, NY, USA, 14 pages.

1 Introduction

AI agents increasingly operate as long-running actors with access to tools, files, shells, browsers, package managers, and external APIs [17, 26, 56]. Coding agents are a prominent example [50, 53]. As agents gain autonomy, they require *AI agent harnesses*: software around the model that improves agent performance while enforcing behavioral constraints for safety, security, and compliance [5].

A central component of such a harness is a policy engine [48]: the part that enforces constraints over the agent's concrete actions. In practice, projects encode many such constraints as behavioral policies in instruction files (`CLAUDE.md`, `AGENTS.md`) [8, 32], so harnesses can provide these instructions to the model directly. Because LLMs comply with instructions probabilistically [25, 31, 40] and may violate them through planning errors, prompt-injection drift, and opaque tool or script behavior [17, 23, 56], harnesses also require deterministic policy engines to enforce compliance at runtime [16, 41, 49].

Many of these policies constrain system-level actions, yet existing policy engines leave a *semantic gap*: *AI agent intent and directives* are expressed in natural language, but enforcement must decide over *system actions.* Our empirical study of 64 projects (§3) finds that 64% of instruction-file statements are behavioral directives, and 83% of those involve system-level behavior: commands executed, files accessed, and network connections made. Existing approaches cannot close this gap. Tool-call-based policy systems [15, 16, 44, 49, 52] in agent runtimes such as Claude Code and Codex, whether LLM guardrails or regex-like checkers, lose track of indirect system actions when agents shell out or spawn subprocesses: the same `git push` can be reached through a tool call, a `bash -c` wrapper, or a compiled binary created in a previous session; and at the repository level, 81% of projects contain cross-event directives that single tool-level interception cannot enforce (e.g., "run tests before commit").

OS-level enforcement with static policy is also insufficient: most directives reference project or task concepts that static rules cannot resolve (e.g., "never modify upstream source," where *upstream source* must be resolved from the repository; "do not update dependencies unless requested"; "the reviewer sub-agent only writes to its project directory"). We find that 74% of system-level directives require project or task context absent from low-level OS events. OS-level enforcement systems and sandboxes for traditional software [6, 11, 14, 47] can observe all system actions but expect pre-defined static policy and return only system events without semantic context, leaving the agent unable to follow its behavioral directives.

We argue that an agent-harness policy engine should be programmable by the agent itself and take effect at OS level: it should let the agent use live project and task context to enforce its own actions and adapt at runtime. Our key observation is that the required context is mainly within the agent itself: the agent must read the repository, interpret the current task, and resolve abstract references into concrete commands and paths before enforcement can begin. This makes the agent the natural producer of concrete policy from its own directives, also reflecting the fact that instruction files are mainly maintained by the agent itself [1, 21] as its persistent memory.

This creates three challenges. First, policy specification must be *expressive* enough to capture directive intent yet *concrete* enough to compile to deterministic kernel checks. Second, enforcement must track policy-relevant state across tools, subprocesses, files, and network endpoints without imposing prohibitive per-syscall overhead. Third, programmability must not become an authority bypass, which requires a different threat model from traditional sandboxes where a trusted administrator authors policy: agents are both policy author and constrained subject, so they need an interface to add constraints without weakening inherited policy set by trusted administrators or parent agents.

AgPlane is a programmable OS-level policy enforcement system for AI agent harnesses. Agents express policies in a compact domain-specific language (DSL) generated from natural-language project directives; AgPlane compiles them into labeled information-flow rules and loads them into an eBPF/BPF-LSM (Linux Security Modules) enforcement engine. Named labels propagate across processes, files, and network endpoints at kernel hooks, encoding execution history as checkable state. When a labeled event matches a rule, AgPlane returns semantic feedback: which rule matched, why, and what compliant alternative satisfies it. Each rule independently selects one of three effects: notify (observe), block (pre-operation denial), or kill (post-operation termination). A layered authority model ensures that higher-authority rules (e.g., platform-level "do not leak secrets") cannot be weakened by agent-authored policy. On a 190-trace decision-compliance benchmark sampled from 38 directives, AgPlane reaches a 75.8% decision-compliance rate, 27.4 percentage points higher than prompt-filter and 30.5 points higher than tool-regex, while adding 1.9% end-to-end overhead on agent workloads (under 8.4% on I/O-intensive builds).

We make three contributions:

1. An **empirical study** of 64 projects that characterizes these gaps (§3).
2. **AgPlane**, a programmable OS-level policy enforcement system that addresses them (§4–5).
3. An **evaluation** on a decision-compliance benchmark, system workloads, and a 20-task OctoBench subset (§6).

2 Background

AI agents and harnesses. AI agents combine model reasoning with external tools and long-running state. They may browse untrusted content, call APIs, manipulate files, execute code, or coordinate workflows across multiple steps [10, 17, 56]. Coding agents such as Claude Code [1], Codex CLI [35], Cursor [2], Devin [13], SWE-agent [53], and OpenHands [50] are a representative deployment domain where these effects are readily observable. Agents operate through an *AI agent harness*: software around the model that maintains the agent loop and session state, routes tool calls, mediates shell, file, and network access, and returns results or feedback to the model, improving agent performance while enforcing behavioral constraints [5, 48]. A single tool invocation can run arbitrary scripts, spawn subprocesses, and touch many files and endpoints before control returns to the model. Projects encode behavioral expectations in instruction files (`CLAUDE.md`, `AGENTS.md`) [7, 8, 32, 42] as *AI agent intent*: natural-language constraints over what the agent should do, avoid, or condition on project and task context. The harness must enforce that intent over *system actions*: concrete commands, file operations, network connections, and subprocess effects.

Information-flow control. Information-flow control (IFC) tracks how data or authority moves from sources to sinks over time. Static IFC systems embed flow constraints in type systems or language annotations [34]. Dynamic IFC and provenance systems attach labels to OS objects and propagate them at runtime: when a process reads a labeled file, the process acquires that label; when it forks, the child inherits it [12, 19, 37, 43]. Later operations are checked against the accumulated label state. OS-level IFC systems such as Flume [28] and HiStar [55] enforce flow policies across processes and files in the kernel, while whole-system provenance frameworks such as PASS [33], Hi-Fi [39], LPM [4], CamFlow [37], and CamQuery [36] record and query cross-event label histories. IFC thus captures constraints whose correctness depends on execution history, which is the same pattern that agent behavioral directives ("run tests before committing") exhibit. More general OS-level mechanisms (seccomp [14], AppArmor [6], Capsicum [51], Landlock [47], BPF-LSM [46], Tetragon [11], KubeArmor [29], and BPF-Contain [20]) also observe or restrict system effects below the tool layer, but require statically pre-written policies and return opaque errors with no connection to the agent's declared directives.

AI agent policy enforcement. We use *runtime policy enforcement system* for the part of an AI agent harness that enforces safety, security, and compliance constraints while the agent executes. Existing enforcement operates at one of three levels. Prompt instructions rely on the model's own compliance, which is inherently probabilistic [25, 31, 40] and vulnerable to indirect prompt injection [17, 23, 56]. Programmable rails and guard agents check prompts, responses, or action trajectories at runtime [10, 41, 52]. Tool-call guardrails and application-level IFC systems [15, 16, 24, 44, 49] intercept at the harness boundary deterministically. All three levels observe only harness-mediated requests, not the system-level effects that follow once a tool starts executing: a subprocess, a shell-out, or a compiled binary can bypass the tool boundary entirely. What remains missing is the connection between AI agent intent and project or task context on one side, and deterministic OS-level observation and enforcement on the other. The empirical study below characterizes how strongly real project instructions demand this connection.

3 Empirical Study

We analyze real agent instruction files to answer a prerequisite question: do project instructions contain behavioral policies that demand OS-level, cross-event enforcement? This section characterizes the directives, classifies their enforcement requirements, and quantifies the context they need to become concrete rules.

3.1 Methodology

We collected public GitHub repositories containing `CLAUDE.md` or `AGENTS.md`, prioritizing the AI-agent ecosystem over filename-only matches and excluding non-code, inactive, fake-star, and stub repositories. The snapshot (2026-05-23 UTC) contains 64 repositories, 84 deduplicated instruction files, and 2,116 extracted statements. A *statement* is a coherent unit expressing one claim, directive, or constraint. A two-pass LLM-assisted pipeline extracted statements with source line ranges and four labels (content type, topic, enforcement level, context requirement); a validation script verified full source coverage and verbatim span matching. A stratified sample of 100 statements was independently reviewed by human annotators, with enforceability labels additionally cross-checked by independent Claude and Codex agents. The following subsections report results along the four labeling axes; E-RQ3 identifies the OS-enforceable subset used in the evaluation (§6). Table 1 collects representative statements that illustrate all four axes; the labels **S1–S8** are referenced throughout.

3.2 E-RQ1: Are Instruction Files Behavioral Policies?

A statement is a *directive* if it asks the agent to perform, avoid, or condition an action; otherwise it is descriptive context. Of the 2,116 statements, 1,361 (64.3%) are directives and 755 (35.7%) are descriptions (Figure 1).

Instruction files are predominantly behavioral policies: the median repository has 71% directives. 75% of repositories have a directive fraction above 50%. At the extremes, one repository is entirely descriptive (0% directives) while another is 97% directives. This majority is invisible at line level: directives average 3.6 lines per statement while descriptions average 6.8 lines, so the same files appear balanced (49% directive) when measured by lines.

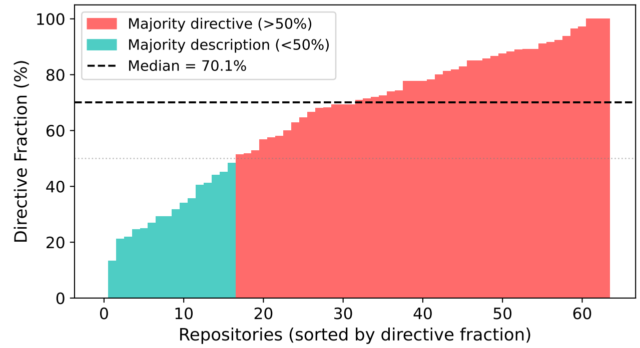

Figure 1. Directive fraction per repository by statement count. Most repositories contain a majority of directive statements.

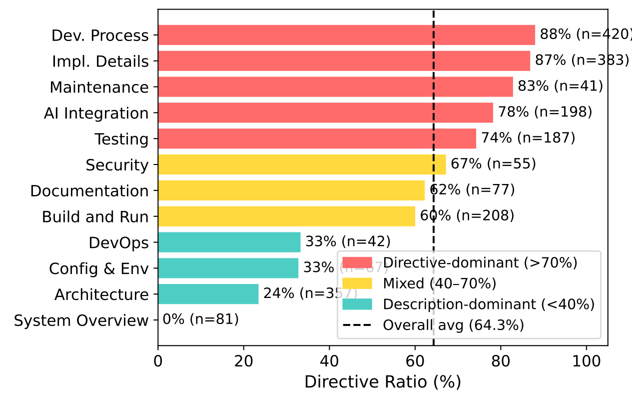

Figure 2. Directive ratio by topic.

3.3 E-RQ2: Which Topics Contain Directives?

Each statement is assigned to one of 12 topic categories adapted from prior instruction-file studies [7], applied at statement granularity rather than file granularity (Figure 2). **Directive density ranges from 0% (System Overview) to 87% (Development Process).** Development Process and Implementation Details are directive-heavy (86.7% and 85.2%, respectively). Architecture is mostly descriptive: directory layouts and design summaries make it one of the largest topics by line count, yet only 23.0% of its statements are directives. A file classified as "Architecture" by prior studies may mostly describe the project, while a shorter Development Process section packs many enforceable rules.

Table 1. Representative corpus statements illustrating all four classification axes. Enforcement level and context requirement apply only to directives (— = not applicable).

	Statement	Type	Topic	Enforcement	Context
S1	"The backend uses Express with TypeScript."	Desc	Architecture	—	—
S2	"Always explain your reasoning before changes."	Dir	AI Integr.	Semantic	—
S3	"Prefer `const` over `let`."	Dir	Impl. Det.	Content	None
S4	"Never push to main directly."	Dir	Dev. Proc.	Per-event	None
S5	"Never modify upstream source code."	Dir	Dev. Proc.	Per-event	Project
S6	"Run the full test suite before committing."	Dir	Testing	Cross-event	Project
S7	"Data read from `.env` must not reach the network."	Dir	Security	Cross-event	Project
S8	"Do not update dependencies without approval."	Dir	Maint.	Per-event	Task

3.4 E-RQ3: Which Directives Require OS-Level Enforcement?

Each directive is classified into the first matching tier of an enforcement waterfall (Figure 3): *semantic-only* (reasoning, communication, or output style), *content* (predicates over file contents), *per-event* (a single command, file access, or network connection), and *cross-event* (history, ordering, lineage, or information flow). We call the union of content, per-event, and cross-event directives *system-observable*; the per-event and cross-event subset that AgPlane can enforce at OS hooks is *OS-enforceable*.

83% of directives constrain system-level actions. Of 1,361 directives, 234 (17.2%) are semantic-only, 520 (38.2%) require content inspection, 392 (28.8%) match a single OS event, and 215 (15.8%) require cross-event state (Table 1, S2–S7 illustrate all four levels). Content inspection alone covers 55.4% of directives; adding per-event matching reaches 84.2%; the remaining 15.8% require cross-event IFC.

Cross-event directives concentrate in Development Process, which accounts for 39.5% of all cross-event directives (Figure 4). Four recurring patterns emerge: temporal ordering ("run tests before committing"), cross-file consistency ("update docs when behavior changes"), multi-step workflows (release checklists with verification steps), and conditional triggers ("if you change specs, also update SDK"). These map directly to IFC primitives: lineage gates, label propagation, and staleness-aware re-arming.

At the repository level, 81% of repositories contain at least one cross-event directive, and 43% require all four enforcement layers.

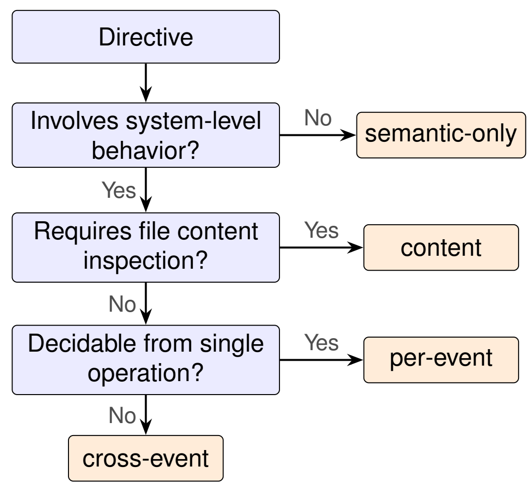

Figure 3. Enforcement-level waterfall. Each directive exits at the first matching tier.

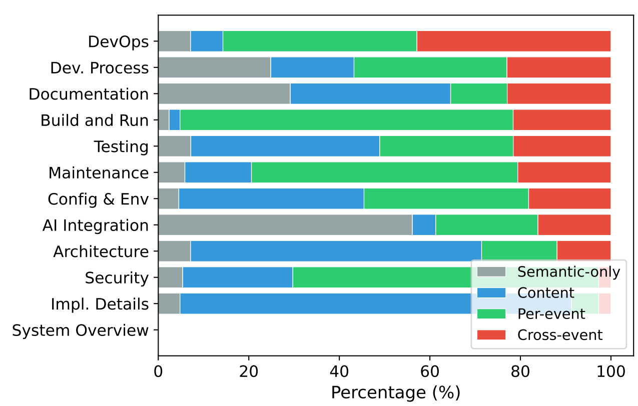

Figure 4. Enforcement profile by topic (normalized). Topics exhibit distinct archetypes; cross-event directives concentrate in Development Process.

3.5 E-RQ4: What Context Is Needed to Instantiate Rules?

Each system-observable directive is classified by what additional context an enforcement mechanism needs beyond the directive text itself (Figure 5): *self-contained* if all commands, paths, and patterns are explicit; *project context* if it references an unresolved repository-specific concept ("the full test suite," "the migration tool"); or *task context* if it depends on the current user request or a per-session grant ("unless explicitly requested," "without approval").

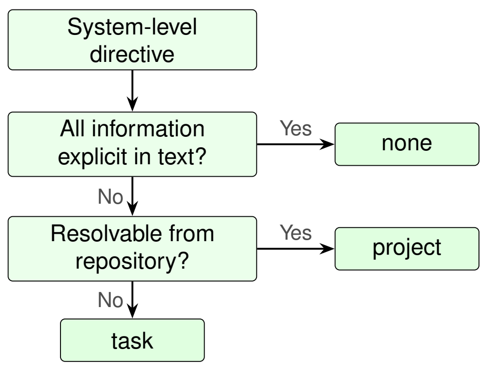

Figure 5. Context-requirement waterfall. Each system-level directive exits at the first matching tier.

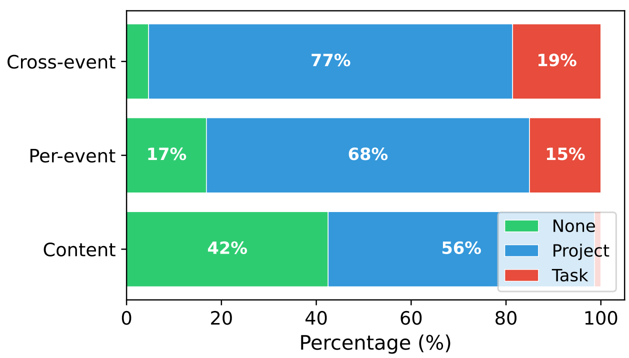

Figure 6. Context requirement by enforcement level. Cross-event directives are 95% context-dependent; content directives are 42% self-contained.

74% of system-observable directives require project or task context to instantiate as concrete rules. Of the 1,127 system-observable directives, only 26.4% are self-contained. Another 64.2% require project context: the directive mentions "the test suite," "the linter," or "upstream source," whose concrete command or path must be resolved from the repository. The remaining 9.4% require task context (Figure 6; Table 1: S4 is self-contained; S5–S7 require project context; S8 requires task context).

Cross-event directives are 95% context-dependent, compared to 58% for content directives. Of cross-event directives, 76.7% require project context and 18.6% task context (95.3% total); content directives are only 57.5% context-dependent. This compounds the enforcement challenge: directives that require cross-event IFC almost never specify the concrete commands and paths needed to write the rule. Statically authored rules can cover only the self-contained fraction; the majority require the agent itself to read the repository, interpret the current task, and produce a concrete policy before enforcement begins.

4 Design

The empirical study establishes four requirements for a policy engine in AI agent harnesses: *agent-defined policy with semantic feedback*, since most rules need project or task context that mainly resides within the agent, and the agent must understand violations to recover; *OS-level cross-event enforcement*, since most directives involve system-level behavior spanning multiple operations, invisible to tool-call guardrails; *safety*, so enforcement holds under planning errors or prompt injection, and agent-authored policy cannot weaken safety constraints; and *low overhead*, so enforcement does not slow the agent's normal workload.

C1: Expressive yet enforceable policy specification (§4.3, §4.4). Instruction-file directives are written over high level repository and task context such as "the test suite," "approved endpoints," or "do not push to main." Kernel enforcement operates over syscalls, file paths, arguments, processes, and endpoints. A specification at the directive level is too ambiguous to enforce deterministically; one at the syscall level is verbose, brittle, and exposes kernel internals that agents and operators cannot reliably audit. Moreover, 81% of repositories require policies spanning multiple operations: "run tests before committing" depends on whether a test execution occurred after the last source change; no single event reveals the full behavior. The DSL must be *concrete* enough to lower to fixed-size kernel predicates, yet *expressive* enough to capture cross-event constraints and preserve the directive's scope, context, and rationale.

C2: Efficient enforcement (§4.4). Cross-event policies require tracking execution history across tools, subprocesses, files, and network endpoints, yet enforcement must avoid reconstructing complete traces on every syscall. The enforcement layer must maintain sufficient policy-relevant state for deterministic checks without prohibitive per-syscall overhead.

C3: Authority-safe agent programmability (§4.5). Traditional sandboxes assume a trusted administrator authors policy and an untrusted process is constrained by it. Here the agent is both policy author and constrained subject: it must adapt policy at runtime (a sub-agent may need a narrower scope, or the current task may require a new constraint), but letting the constrained actor write or specialize policy introduces a privilege-weakening risk. Agents and sub-agents may add rules, specialize targets, or narrow scope, but must never weaken platform or project constraints.

4.1 Threat Model

Given the dual role described in C3, AgPlane targets a *cooperative but unreliable* agent: its original intent is honest, but execution may diverge due to planning errors, shell-script side effects, opaque tool behavior, or prompt injection [38]. AgPlane does not detect prompt injection; it enforces loaded rules independently of agent intent, providing defense-in-depth for actions that rules cover. The adversary can control prompt content, tool outputs, browser pages, and API responses, inducing the agent to issue arbitrary tool calls or shell commands; the adversary cannot compromise the kernel, the eBPF loader, the runner, or higher-authority policy blobs.

Security properties. The adversary may cause the agent to exfiltrate sensitive data, modify out-of-scope files, execute forbidden commands, or bypass required workflow steps. Given correctly authored rules that cover these actions, AgPlane enforces two kernel-level properties: (1) *monotonic authority*: agent-authored rules can only tighten, never weaken, inherited higher-authority constraints; and (2) *mediation completeness*: no event delivered to a configured AgPlane hook bypasses the rule check (hooks include exec, open, read, write, unlink, connect, and fork, among others). Mediation guarantees that every hooked operation is checked, not that all policy-violating behaviors are caught: detection depends on rule coverage and accumulated label state. Block effects are synchronous at pre-operation hooks (no TOCTOU gap). Kill effects are post-operation: the triggering syscall completes and its side effect may already be visible, but the process is terminated before any subsequent monitored action; exfiltration-preventing rules should therefore use block. Notify effects are observation-only.

Trust boundaries. Higher-authority policies (platform, project) are authored by trusted principals. Arbitrary commands are in scope with respect to policies expressed over monitored events; AgPlane covers alternate execution paths (subprocesses, shell wrappers, compiled binaries) that manifest as hooked OS events. Semantic equivalents outside the policy vocabulary are out of scope (e.g., a custom Git-protocol client bypassing `git push`).

The trusted computing base (TCB) consists of the eBPF enforcement engine and pre-compiled higher-authority policy blobs, assumed signed and loaded through a trusted path; the loader is trusted for this launch phase. Security properties hold once policy is loaded and the target process enters the monitored domain. The compiler and runner handle only agent-authored rules and are outside the TCB: bugs in these components can weaken the agent's own constraints but cannot affect higher-authority policy. The monitored domain is the agent's entire process tree. Kernel compromise, root or `CAP_BPF` compromise, deliberate side channels, and semantic content checks not observable at syscall boundaries (e.g., whether a written file contains a credential) are out of scope.

4.2 System Overview

AgPlane addresses these challenges with three abstractions (Figure 7): a policy DSL (§4.3) that compiles directive-level and cross-event constraints into kernel-level IFC tables (C1); a labeled information-flow model (§4.4) that summarizes execution history as compact object labels (C1, C2); and a layered

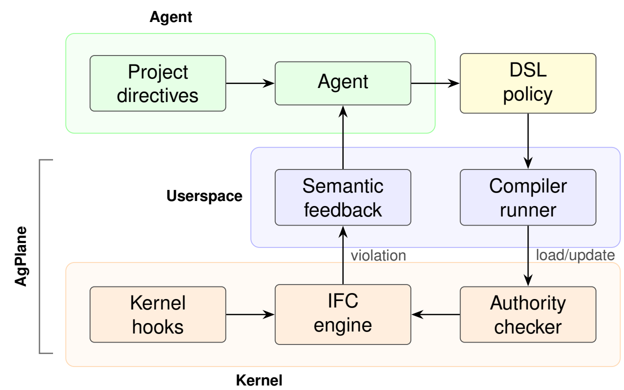

Figure 7. AgPlane architecture: policy generation, compilation, kernel enforcement, and feedback loop.

authority model (§4.5) that ensures monotonic tightening (C3). When a rule fires, AgPlane returns semantic feedback: the rule name, the author's reason, and a remediation path.

4.3 Policy DSL

The DSL bridges AI agent intent and enforceable system actions. A policy contains *sources* that introduce labels, *rules* that match labeled events at sinks, and optional *gates* that express cross-event ordering constraints. The DSL restricts expressiveness to source/sink/gate patterns that compile to fixed-size rodata, avoiding direct exposure of BPF hooks, maps, label bits, or verifier constraints.

Sources bind labels to system objects: an `exec` source labels processes that execute a matching binary; a `file` source labels matching paths; a network source labels matching endpoints. Rules bind an effect (`notify`, `block`, or `kill`) to an operation (`exec`, `open`, `read`, `write`, `unlink`, or `connect`). Conditions express common agent workflow constraints: a target allow-list, a lineage gate, or a temporal `after ... since ...` ordering gate.

Figure 8 illustrates the three main rule shapes. The first is per-event: if the agent process pushes to `main`, the event matches immediately. The second is cross-event: committing is compliant only if the required test command ran after the most recent source write. The third uses the same temporal gate for authority exceptions: force-pushing is blocked unless the agent first writes a declaration file, which higher-authority policy can further constrain.

The DSL is intentionally narrower than a general mandatory access-control language. It targets the policy shapes that recur in agent instruction files: prohibit a command, restrict writes to a scope, block a sink after a sensitive read, require a validation step before a release action, and route sensitive data through an approved tool. In our evaluation, an LLM translation agent successfully translates all 607 OS-enforceable directives from the corpus into compilable DSL

```
source AGENT = exec "claude"
source SECRET = file "**/.env"

# Per-event: from CoplayDev/unity-mcp
rule no-push-main:
  kill exec "git" "push" "main" if AGENT
 because "Do not push to main; open a PR instead."

# Cross-event: from chenhg5/cc-connect
rule tests-before-commit:
  block exec "git" "commit"
    if AGENT unless after exec "go"
      since write "**/*.go"
 because "Run `go test ./...` before committing."

# Authority exception gate
rule no-force-push:
  block exec "git" "push" "--force"
    if AGENT unless after write
      ".agent/force-push-necessary"
 because "Write a justification before force-pushing."
```

Figure 8. Three AgPlane rules drawn from real projects: a per-event rule, a cross-event ordering gate, and an authority exception gate.

rules (§6). The compiler lowers these patterns into fixed-size kernel tables; policy authors need not reason about BPF maps, verifier constraints, or binary layout.

4.4 Labeled Information-Flow Model

AgPlane represents policy state as labels on OS objects. Processes, files, and network endpoints each carry a 64-bit label mask, enabling cross-boundary flow policies (e.g., "data read from `.env` must not reach a network endpoint"). A source declaration adds labels when an object matches the source pattern. Let $L(x)$ denote the label mask of object x. Labels propagate through observed system-level edges:

- **fork**$(p \to c)$: $L(c) := L(c) \cup L(p)$.
- **exec**(p, f): $L(p) := L(p) \cup L(f) \cup S(f)$, where $S(f)$ is the set of source labels matching f.
- **read**(p, f): $L(p) := L(p) \cup L(f)$.
- **write**(p, f): $L(f) := L(f) \cup L(p)$.
- **connect**(p, e): $L(e) := L(e) \cup L(p)$.

These transfer functions maintain a key invariant: if an object carries label ℓ, then information from a source tagged with ℓ has reached that object through observed execution edges. Rules check whether an event's subject or target carries required labels, carries forbidden labels, or satisfies a temporal gate.

The model operates at object granularity rather than byte granularity, trading precision for low-overhead whole-session coverage. This is a deliberate fit for AI agent harnesses: repository instructions refer to commands, files, directories, endpoints, and workflow steps, not individual bytes. Object-level labels capture cross-event compliance patterns that tool-call filters miss, without incurring the complexity of byte-level taint tracking [27].

For efficiency, labels summarize history rather than recording traces. Each object stores a fixed-size mask, and each rule checks masks and bounded predicates at the current event. This keeps cross-event enforcement on the syscall path without log reconstruction or unbounded history queries.

Labels are monotonic: propagation adds labels but never removes them. Once an object carries a label, every subsequent consumer inherits it. This ensures that cross-event constraints remain checkable for the entire session without requiring explicit label cleanup.

4.5 Layered Policy Authority

AgPlane must let agents adapt policy at runtime without weakening inherited constraints.

AgPlane organizes policy around a *domain hierarchy*. A domain is the runtime policy boundary for a process tree: every pid maps to a domain, and every domain maps to an effective policy set. The core data model is:

```
pid    -> domain
domain -> parent + policy(domain)
policy(D) = policy(parent(D)) + local(D)
```

Policy evolution is *monotonically tightening*: a child domain inherits all parent rules and may add local rules, labels, or gates, but cannot remove, disable, or weaken any inherited rule. When an agent forks a sub-agent, the child process enters a child domain that inherits the parent's full policy; the sub-agent may add further restrictions but cannot shed inherited constraints.

The root domains correspond to platform/operator and project-maintainer policy; the agent creates session and sub-agent domains below them. Higher-authority rules that admit exceptions use the same temporal gate as cross-event workflow rules (Figure 8, `no-force-push`): the agent satisfies the gate by creating a declaration file, which the IFC engine tracks like any other label-propagating event. Higher-authority policy can further constrain which processes may write the declaration file, keeping exception paths under platform or project authority.

The prototype resolves the initial domain hierarchy at compile or load time. Runtime policy additions (sub-agent restrictions, task-specific rules) are submitted as precompiled deltas through a userspace ring buffer. The kernel derives the submitting domain from the caller's pid-to-domain mapping, not from user-provided metadata, and admits updates only to the submitting domain or its descendants. The kernel authority checker admits a delta only if it satisfies a partial order: it may introduce fresh local labels, bind new rules, or narrow target scope, but it cannot modify inherited labels, sources, gates, exception predicates, or rule effects; nor can it remove inherited rules or widen scope. A delta cannot create labels that satisfy an inherited exception gate unless higher-authority policy explicitly defines that path. Accepted deltas are merged into the domain's effective policy; the kernel does not parse configuration or resolve project context on the event path.

5 Implementation

AgPlane consists of a Rust compiler/runner (3.2 K LoC) and an eBPF enforcement engine (1.8 K LoC of BPF C). The compiler parses the DSL, allocates label bits, lowers Boolean label expressions and path/network patterns, and emits a fixed-size `#[repr(C)]` configuration blob consumed directly by the eBPF loader. Human-readable rule names and reasons stay in userspace metadata; the kernel receives only compact rule tables and emits rule identifiers when events match.

The eBPF engine attaches to process, file, and network hooks via BPF-LSM (pre-operation enforcement) and tracepoints (observation and post-operation termination). It maintains label state in per-object BPF maps, applies the transfer functions from §4, and evaluates the compiled rules on each observed event. Userspace does not re-detect violations: it loads the policy, starts the target process, and formats kernel-reported matches into semantic feedback.

The runner (`AgPlane run -- <cmd>`) discovers the project policy file, compiles and loads it before the target starts, seeds the target process tree with the agent label, and records kernel matches in a feedback file. Higher-authority policy blobs are pre-compiled and loaded through the same path; the runner treats them as opaque signed inputs. Compatible agent hooks relay this feedback into the model's next turn. The runner is intentionally thin: policy decisions remain in the kernel; the runner mediates between the compiled policy, the target process, and agent-facing diagnostics.

The domain hierarchy and authority checker are implemented in the eBPF engine. Each domain is a BPF map entry storing a parent pointer, an inherited-rule mask, and an inherited-label mask. Each pid-to-domain mapping is stored in a separate BPF map. When a runtime delta arrives through the userspace ring buffer, the checker looks up the submitting process's domain via `bpf_get_current_pid_tgid()`, verifies the target domain is the submitter's own or a descendant, and validates the delta against the inherited masks: new rules and labels are admitted only if they do not clear any inherited-mask bit or modify any inherited rule effect, source, gate, or exception predicate. Weakening deltas are rejected before they reach the effective policy.

6 Evaluation

We evaluate AgPlane along three evaluation research questions (RQ1–RQ3, distinct from the empirical E-RQs in §3).

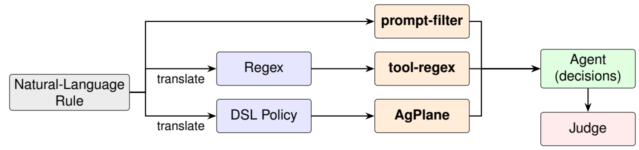

Figure 9. RQ1 evaluation pipeline: three enforcement paths from natural-language rule to agent-level decision.

RQ1 measures end-to-end policy compliance on directive-derived rules from the empirical corpus; RQ2 quantifies per-event and end-to-end overhead; RQ3 tests real-world coding tasks on an OctoBench subset with the official judge.

- **RQ1 (Policy compliance)** Compared with prompt-filter, tool-layer, and feedback-ablation baselines, does AgPlane improve policy compliance for directive-derived rules across direct and bypass execution paths?
- **RQ2 (Overhead)** What is the per-event and end-to-end overhead of AgPlane's label propagation and rule checking?
- **RQ3 (Real-world coding tasks)** On complete coding-agent tasks in real repositories, does AgPlane's OS-level enforcement and semantic feedback improve policy compliance and task success rate?

6.1 Experimental Setup

All experiments run on a single machine with an Intel Core Ultra 9 285K CPU (24 hardware cores), 125 GiB RAM, Linux 6.15.11, ext4 filesystem, and libbpf 1.x with `bpf_loop` support. The live enforcement runs require BPF-LSM active (`bpf` in `/sys/kernel/secur` the RQ2 overhead artifacts record the exact LSM state used for each performance run.

6.2 RQ1: End-to-End Policy Compliance

Benchmark Motivation. We build a new trace-level benchmark using similar methodology to existing benchmarks, but designed to challenge runtime enforcement systems with bypass paths invisible to tool-call interception. Existing enforcement benchmarks such as ShieldAgent-Bench [30] and ToolSafe [9] evaluate guardrail precision and recall at the tool-call or trajectory level. Runtime enforcement evaluations in AgentSpec, Progent, FIDES, and CaMeL [15, 16, 44, 49] measure attack-success-rate reduction on AgentDojo scenarios [17]. However, all operate at the tool-call boundary: they do not test whether enforcement holds when an agent reaches the same system effect through a subprocess, a shell script, or a compiled binary.

Rule set and scope. The empirical study identifies 607 OS-enforceable directives: 392 per-event and 215 cross-event. RQ1 uses this subset because these directives can be represented at process, file, or network hooks. Semantic-only and content directives are excluded: the former have no system-observable counterpart; the latter require file-content inspection rather than OS-level IFC. We evaluate 38 randomly-selected statements, sampling 2–3 from each of 15 repositories to cover both per-event and cross-event rules, project- and task-context requirements, and common repository policy themes. The final manifest contains 20 per-event and 18 cross-event rules, proportional to the split in the full corpus.

Traces. Each trace is a JSONL transcript generated by Codex CLI under GPT-5.5 that simulates real-world agent task steps, including user prompts and concrete tool calls that establish the repo state. A separate *trace generator* reads the directive and project structure to produce five traces per statement. For each statement we construct two compliant traces and three violation traces. Allowed-effect traces exercise explicitly permitted system-level behavior close to the prohibited action; lookalike traces perform operations superficially similar to the prohibited action but technically compliant. Visible violation traces directly trigger the prohibited operation; script-mediated traces split script authoring and execution across separate tool calls; opaque traces expose only a neutral helper entrypoint while the real side effect is visible only at the OS level. This yields 190 traces and 760 system–trace cells across four systems: prompt-filter, tool-regex, AgPlane without structured feedback (AgPlane-opaque), and full AgPlane. In the primary model setting, we replay them on codex-sdk with Qwen3.6-27B as the LLM: the replay executes each tool call on a copy-on-write repository image, and the tested agent continues for a bounded number of real tool steps. We repeat the full RQ1 pipeline with DeepSeek-Pro V4 for the LLM-dependent roles: tested-agent continuation, the prompt-filter classifier, and trajectory judging. Ground truth and policy artifacts are not shown to the tested agent.

Compared systems. RQ1 compares four runtime policy enforcement systems on the same agent traces (Figure 9): a prompt-based action filter, a tool-input regex policy, AgPlane without structured corrective feedback (AgPlane-opaque), and full AgPlane. The prompt and regex baselines instantiate two common runtime enforcement layers: LLM/prompt-based rails over proposed actions [41, 52] and deterministic tool-call policies over tool names and arguments in the style of AgentSpec [49]. The opaque ablation isolates the value of semantic feedback on top of OS-level observation. A translation agent (Codex CLI [35] with GPT-5.5) reads the sampled directive text, the project's `CLAUDE.md`, and the cloned repository, then produces the DSL rule for AgPlane or the regex for the tool-regex baseline; it does not have access to the ground-truth traces. For AgPlane, the agent iteratively invokes the compiler: if a rule fails to compile, it observes the diagnostic, revises, and retries until compilation succeeds. The prompt-filter baseline does not require translation; it directly decides whether to allow or block each proposed tool call based on the directive text and the proposed tool input. When a violation is detected, the harness returns structured feedback to the agent, which can use this information to adjust its behavior and comply with the directive in subsequent steps. Figure 8 shows representative AgPlane rules.

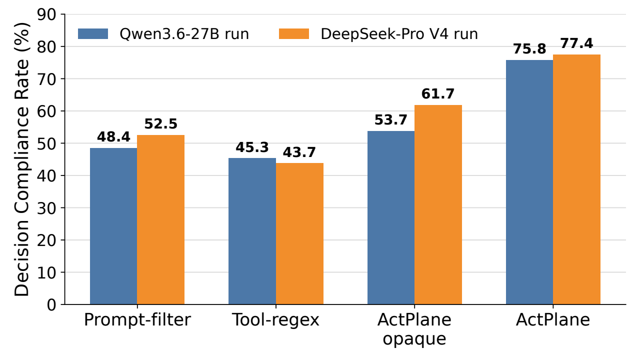

Figure 10. Overall RQ1 Decision Compliance Rate across 190 traces under two end-to-end model settings. In each setting, the tested agent, prompt-filter classifier, and trajectory judge use the indicated model.

Metric. A separate LLM trajectory judge assigns TP, TN, FP, or FN from the raw runner results. The primary RQ1 result uses the Qwen3.6-27B model setting above; the DeepSeek-Pro V4 result is a full model-setting replication, not a judge-only rescore. In each setting, the judge sees the ground truth, whether the policy enforcement system triggered and its feedback, and what the agent did in subsequent steps. TP means a violation trace received enforcement-visible intervention or agent awareness; TN means a compliant trace completed without wrongful intervention; FP means a compliant trace was wrongly blocked or steered; FN means a violation trace was missed. We report *Decision Compliance Rate* (DCR), the fraction of trace-conditioned decision points whose final outcome is policy-correct: DCR = (TP + TN)/(TP + TN + FP + FN). DCR is end-to-end from natural language directive to agent behavior: it includes policy translation, runtime enforcement, feedback delivery, and agent recovery. All 760 system–trace cells and all 760 primary trajectory-judge cells are present, with no judge parse errors. If a model-setting run yields unclear, we exclude that cell from the DCR denominator and report the unclear count in the artifact table; the DeepSeek-Pro V4 run marked 39 of 760 cells unclear.

Results. AgPlane reaches a DCR 27.4 percentage points higher than prompt-filter and 30.5 points higher than tool-regex. The difference comes primarily from violation recall. Full AgPlane reduces FNs to 28, compared with 70 for prompt-filter and 76 for tool-regex, because it observes runtime side effects below the tool layer and can detect script- and fixture-mediated violations that are not available to prompt or raw

Table 2. RQ1 confusion matrix by runtime policy enforcement system under the primary Qwen3.6-27B model setting.

System	DCR	TP	TN	FP	FN	Judged
PROMPT-FILTER	48.4%	44	48	28	70	190
TOOL-REGEX	45.3%	38	48	28	76	190
AGPLANE-OPAQUE	53.7%	27	**75**	**1**	87	190
AgPlane	**75.8%**	**86**	58	18	**28**	190

tool-input filters. It also reduces false positives on compliant traces relative to the baselines, though this margin is smaller: full AgPlane has 18 FPs, compared with 28 for both prompt-filter and tool-regex. The DeepSeek-Pro V4 end-to-end run in Figure 10 preserves the same ordering on the same 760 system–trace cells: AgPlane remains highest at 77.4% DCR, followed by AGPLANE-OPAQUE at 61.7%, prompt-filter at 52.5%, and tool-regex at 43.7%.

The feedback ablation isolates the value of semantic recovery information. AGPLANE-OPAQUE enforces the same kernel-level rules but returns no structured feedback: the agent sees only a raw `EPERM` or process termination. Without a feedback channel, the system never actively communicates with the agent on compliant traces, which explains its near-zero FP (1) and high TN (75), not because enforcement is more precise, but because the agent receives no enforcement signal on compliant traces. Conversely, on violation traces the kernel does block the offending syscall, yet the agent cannot diagnose the cause or recover, so the judge classifies most blocked violations as FN (87 FNs). Full AgPlane opens a structured feedback channel: rule name, reason, and remediation reach the agent after every match. This raises violation recall substantially (FN drops from 87 to 28, TP rises from 27 to 86) but introduces a tradeoff on compliant traces: the active feedback channel occasionally triggers agent overcorrection on boundary cases, raising FP from 1 to 18 and lowering TN from 75 to 58. The net effect is positive (correct responses rise from 102 to 144) because the mechanism is not additional detection alone; the feedback channel converts kernel-level observation into agent-level recovery.

AgPlane's advantage concentrates on violation traces that bypass the tool boundary (Figure 11). On the two compliant families (allowed-effect and lookalike), all systems except AGPLANE-OPAQUE show comparable performance; AGPLANE-OPAQUE's near-zero FP rate gives it the highest compliant-trace DCR. The difference emerges on violation families. On directly visible violations, prompt-filter and tool-regex detect some cases, but AgPlane detects substantially more because it observes the actual system effect rather than the proposed tool input. On script-mediated and opaque-fixture violations, prompt-filter and tool-regex largely fail: tool-call interception cannot see effects that occur inside a subprocess or behind a neutral entrypoint. AgPlane retains detection through kernel-level label propagation, which follows

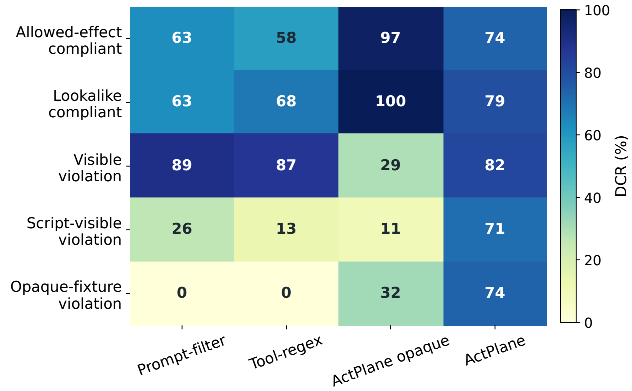

Figure 11. RQ1 breakdown by trace family. Cells show DCR (%) for each system×family; darker is higher.

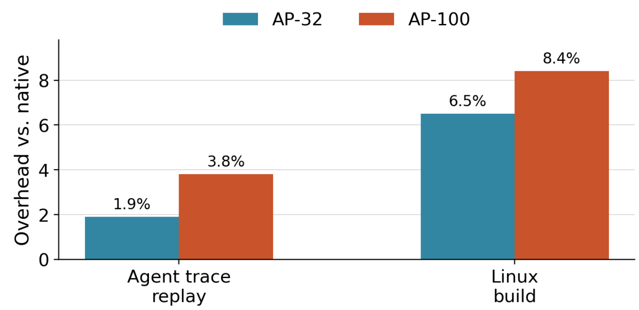

Figure 12. End-to-end overhead normalized to native execution.

the effect regardless of how the agent invoked it. The feedback ablation confirms that detection alone is insufficient: AGPLANE-OPAQUE observes the same kernel events but produces low violation-family DCR because the agent cannot recover without a structured corrective payload.

Authority invariant. To validate monotonic authority (§4.5), we submit a suite of policy deltas containing 20 test cases to the kernel authority checker: narrowing deltas (additional rules, tighter scope) are accepted, while weakening deltas (removing inherited rules, widening scope, modifying inherited effects or gates) are rejected. All test cases pass.

6.3 RQ2: Overhead

6.3.1 Macrobenchmarks (End-to-End Workloads). We measure end-to-end overhead on two deterministic workloads under no-hit AgPlane configurations: (1) an agent trace suite that replays 20 corpus-derived traces (68 tool actions, 20 Bash subprocesses), and (2) a Linux kernel build (`defconfig` + `vmlinux`, `make -j24`, clean output directory). Each workload runs three trials per configuration. Fixed workloads eliminate LLM inference variance and isolate AgPlane's runtime cost.

Table 3. Per-operation latency in microseconds for no-hit configurations. Values are medians across seven trials.

Operation	Native p50	AP-1 p50	AP-32 p50	AP-100 p50
`fork`	48.94	52.06	74.05	69.33
`exec`	248.30	263.95	314.86	317.03
`open`	0.58	1.24	6.92	13.40
`write`	0.27	0.78	0.79	0.84
`connect`	0.58	0.61	1.98	3.17

Results. At 32 active no-hit rules, AgPlane adds 1.9% overhead on the agent-trace replay and 6.5% on the Linux kernel build. At 100 rules, overhead remains below 8.4%. The agent-trace workload exhibits lower overhead because tool actions are interspersed with model inference pauses that dwarf syscall cost; the kernel build stresses sustained I/O and process creation, exposing more of AgPlane’s per-event cost.

6.3.2 Microbenchmarks (Per-Syscall Latency). We measure AgPlane’s per-event latency for five syscall types (`fork`, `exec`, `open`, `write`, `connect`) under five no-hit configurations: native, and AgPlane with 1, 10, 32, and 100 active rules (AP-N denotes AgPlane with N rules). Each configuration $\times$ syscall type runs 10K–100K iterations pinned to a single CPU core. For each trial we compute p50 and p99 latencies; Table 3 reports the median across seven trials.

Results. Per-call overhead is dominated by process operations, while file and network operations add single-digit microseconds (Table 3). `fork` and `exec` incur the highest absolute added cost (3–69 µs at AP-100) but remain modest relative to their native latency because process-management and scheduler latency dominates the baseline. File operations (`open`, `write`) and `connect` are sub-microsecond natively; AgPlane adds a path-hash lookup and rule scan, raising `open` to 13.4 µs at AP-100.

Despite these per-call additions, the macrobenchmark impact remains low (1.9%–8.4%) because absolute added cost is small relative to the computation, I/O, and LLM inference time that dominates agent workloads. The cumulative AgPlane overhead of an entire tool-call’s syscall sequence is 5–6 orders of magnitude smaller than a single LLM inference turn (2–10 s). All overhead measurements use no-hit configurations, which exercise steady-state label propagation and rule scanning without violation reporting or userspace feedback formatting.

6.4 RQ3: Real-World Coding Tasks (OctoBench)

Goal. Unlike RQ1, which isolates single decision points, RQ3 evaluates AgPlane on complete coding-agent tasks in real repositories, scored by the official OctoBench checklist judge.

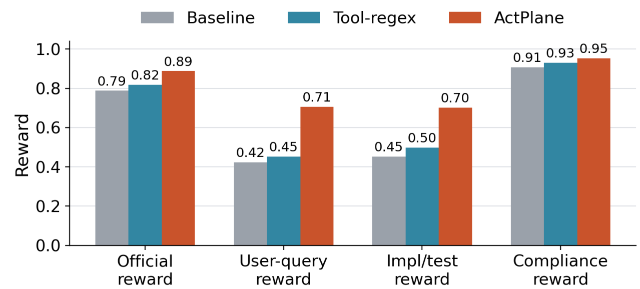

Figure 13. OctoBench 20-task subset: reward breakdown by system.

Benchmark and subset. OctoBench [18] is a repository-grounded coding benchmark with 217 Dockerized tasks spanning system prompts, tool schemas, project files (`CLAUDE.md`, `AGENTS.md`), and user queries. We use the official evaluator unchanged. Because AgPlane cannot observe purely semantic checks (e.g., tone), we select a 20-task OS-observable subset. A task is eligible when at least one checklist item maps to a concrete command, file operation, generated artifact, or tool-execution constraint that can be expressed as a case-specific policy before execution. The selected subset spans `CLAUDE.md`, `AGENTS.md`, and user-query policy sources, two agent scaffolds, and seven Docker images. Pure style, tone, and natural-language-only cases are excluded; the remaining 197 tasks lack an OS-observable checklist item (e.g., “respond in a friendly tone”) and fall outside AgPlane’s enforcement scope. Each task runs under three conditions: baseline (no enforcement), tool-regex (case-specific tool-call hooks), and AgPlane (OS-level hooks plus semantic feedback).

Metrics. The primary metric is official OctoBench reward (fraction of checklist policies judged satisfied). We additionally report three diagnostic submetrics from the same checklist results: *user-query reward* (user-task checks only), *implementation/test reward* (implementation, testing, configuration, and modification checks), and *compliance reward* (compliance-typed checks). All submetrics use the official LLM-based checklist judge.

Results. On the 20-task subset, official reward is 0.788 (baseline), 0.818 (tool-regex), and 0.888 (AgPlane): a 10.0-point gain over baseline and 7.0 points over tool-regex. The improvement is not limited to compliance-typed checks: user-query reward rises by 28.2 points and implementation/test reward by 25.0 points over baseline. These results support the claim that OS-level enforcement with semantic feedback improves policy compliance on real coding-agent tasks. We note that OctoBench uses an LLM checklist judge rather than deterministic test scripts, so we do not claim deterministic task completion.

7 Related Work

In-kernel provenance and information flow. Whole-system provenance systems such as PASS [33], Hi-Fi [39], LPM [4], and CamFlow [37] record audit-grade provenance graphs via LSM hooks but do not compile agent-oriented policy or return semantic feedback. CamQuery [36] is the closest prior system: it evaluates provenance queries inline in the kernel, propagates labels across OS objects, and can deny matching operations. AgPlane shares this label-propagation model but targets cooperative AI agents rather than adversarial intrusions, compiles an agent-facing source/sink DSL rather than general provenance queries, and returns semantic feedback to the matching actor. Dynamic taint-tracking systems [12, 19, 27, 54] operate at byte or instruction granularity for vulnerability detection; AgPlane operates at object granularity, trading precision for low-overhead whole-session coverage.

eBPF enforcement and agent policy systems. BPF-LSM [45, 46] lets eBPF programs attach to security hooks and return allow/deny verdicts; AgPlane uses it as its pre-operation enforcement backend. Contemporary eBPF tools such as Tetragon, Tracee, and eBPF-PATROL [3, 11, 22] provide per-event predicates or single-channel lineage flags; AgPlane propagates multi-label information flow across channels and checks cross-event conditions. At the agent layer, programmable rails and guard agents check prompts or action trajectories [41, 52], and AgentSpec [49] enforces deterministic guards at the tool-call boundary; AgPlane complements them below the tool API, where shell-outs and subprocesses remain visible. FIDES and CaMeL [15, 16] apply typed IFC within the agent loop, which catches data-flow violations at the API boundary but cannot observe effects that escape through shell-outs, subprocesses, or compiled binaries; they also do not expose a layered authority stack that separates platform, project, and agent policy. Flume [28] provides decentralized labels with explicit capabilities but requires per-application integration rather than a compile-time authority hierarchy. AgPlane applies information-flow reasoning at the OS boundary with a layered authority model, where observation persists across context windows and cannot be circumvented by subprocess indirection.

Agent instruction files. Recent studies characterize `CLAUDE.md` and `AGENTS.md` files along dimensions of content taxonomy, maintenance practices, and task efficiency [7, 8, 32, 42]. These works classify at file or section-heading granularity. Our empirical study (§3) classifies at statement level along four axes, specifically asking which instructions require cross-event enforcement and which need project or task context to instantiate.

8 Conclusion

Agent instruction files are behavioral policies, and most of them constrain system-level actions that span multiple events. AgPlane makes these policies enforceable by compiling a compact DSL into labeled information-flow rules that run in an eBPF/BPF-LSM engine below the tool layer, and by returning semantic feedback that enables agent recovery. On a 190-trace benchmark the system achieves a 75.8% decision-compliance rate, 27–30 percentage points above prompt-filter and tool-regex baselines, while adding less than 2% end-to-end overhead. The current scope is limited to OS-observable directives (45% of the corpus); content-inspection and semantic-only directives require complementary mechanisms. By placing enforcement at the syscall boundary with agent-facing feedback, AgPlane provides a foundation for harness-level policy that remains effective regardless of how tools, subprocesses, or sub-agents invoke system operations.

References

[1] Anthropic. 2025. Claude Code. https://docs.anthropic.com/en/docs/claude-code.

[2] Anysphere. 2025. Cursor: The AI Code Editor. https://cursor.com/.

[3] Aqua Security. 2026. Tracee: Linux Runtime Security and Forensics using eBPF. https://github.com/aquasecurity/tracee.

[4] Adam Bates, Dave Tian, Kevin R. B. Butler, and Thomas Moyer. 2015. Trustworthy Whole-System Provenance for the Linux Kernel. In *24th USENIX Security Symposium (USENIX Security 15)*. USENIX Association, Washington, DC, 319–334. https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/bates

[5] Birgitta Boeckeler. 2026. Harness Engineering for Coding Agent Users. https://martinfowler.com/articles/harness-engineering.html.

[6] Canonical Ltd. 2024. AppArmor Security Profiles. https://apparmor.net/.

[7] Worawalan Chatlatanagulchai, Hao Li, Yutaro Kashiwa, Brittany Reid, Kundjanasith Thonglek, Pattara Leelaprute, Arnon Rungsawang, Bundit Manaskasemsak, Bram Adams, Ahmed E. Hassan, and Hajimu Iida. 2025. Agent READMEs: An Empirical Study of Context Files for Agentic Coding. arXiv:2511.12884. https://arxiv.org/abs/2511.12884

[8] Worawalan Chatlatanagulchai, Kundjanasith Thonglek, Brittany Reid, Yutaro Kashiwa, Pattara Leelaprute, Arnon Rungsawang, Bundit Manaskasemsak, and Hajimu Iida. 2025. On the Use of Agentic Coding Manifests: An Empirical Study of Claude Code. arXiv:2509.14744. https://arxiv.org/abs/2509.14744

[9] Jielin Chen et al. 2026. ToolSafe: Towards Agent Safety by Step-Level Tool Invocation Safety Detection. arXiv:2601.10156

[10] Sahana Chennabasappa, Cyrus Nikolaidis, Daniel Song, David Molnar, Stephanie Ding, Shengye Wan, Spencer Whitman, et al. 2025. LlamaFirewall: An Open Source Guardrail System for Building Secure AI Agents. arXiv:2505.03574. https://arxiv.org/abs/2505.03574

[11] Cilium Project. 2026. Tetragon: eBPF-based Security Observability and Runtime Enforcement. https://tetragon.io/.

[12] James Clause, Wanchun Li, and Alessandro Orso. 2007. Dytan: A Generic Dynamic Taint Analysis Framework. In *Proceedings of the 2007 International Symposium on Software Testing and Analysis*. Association for Computing Machinery, London, United Kingdom, 196–206. doi:10.1145/1273463.1273490

[13] Cognition. 2025. Devin: The First AI Software Engineer. https://devin.ai/.

[14] Jonathan Corbet. 2015. A seccomp overview. https://lwn.net/Articles/656307/.

[15] Manuel Costa, Boris Koepf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Beguelin. 2025. Securing AI Agents with Information-Flow Control. arXiv:2505.23643. https://arxiv.org/abs/2505.23643

[16] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramer. 2025. Defeating Prompt Injections by Design. arXiv:2503.18813. https://arxiv.org/abs/2503.18813

[17] Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramer. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems*. https://proceedings.neurips.cc/paper_files/paper/2024/file/97091a5177d8dc64b1da8bf3e1f6fb54-Paper-Datasets_and_Benchmarks_Track.pdf

[18] Yangruibo Ding, Siyuan Cheng, Xiaohan Wang, Yifan Song, Yiming Wang, Markus Mock, and Chenguang Wang. 2026. OctoBench: Benchmarking Scaffold-Aware Instruction Following in Repository-Grounded Agentic Coding. arXiv:2601.10343. https://arxiv.org/abs/2601.10343

[19] William Enck, Peter Gilbert, Byung-Gon Chun, Landon P. Cox, Jaeyeon Jung, Patrick McDaniel, and Anmol N. Sheth. 2010. TaintDroid: An Information-Flow Tracking System for Realtime Privacy Monitoring on Smartphones. In *9th USENIX Symposium on Operating Systems Design and Implementation (OSDI 10)*. USENIX Association, Vancouver, Canada, 393–407. https://www.usenix.org/conference/osdi10/taintdroid-information-flow-tracking-system-realtime-privacy-monitoring

[20] William Findlay, David Barrera, and Anil Somayaji. 2021. BPFContain: Fixing the Soft Underbelly of Container Security. arXiv:2102.06972. https://arxiv.org/abs/2102.06972

[21] Matthias Galster, Sebastian Baltes, and Christoph Treude. 2026. Configuring Agentic AI Coding Tools. *arXiv preprint arXiv:2602.14690* (2026).

[22] Sangam Ghimire, Nirjal Bhurtel, Roshan Sahani, and Sudan Jha. 2025. eBPF-PATROL: Protective Agent for Threat Recognition and Overreach Limitation using eBPF in Containerized and Virtualized Environments. arXiv:2511.18155. https://arxiv.org/abs/2511.18155

[23] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*. Association for Computing Machinery, 79–90. doi:10.1145/3605764.3623985

[24] Invariant Labs. 2025. Invariant Guardrails Documentation. https://explorer.invariantlabs.ai/docs/guardrails/.

[25] Yuxin Jiang, Yufei Wang, Xingshan Zeng, Wanjun Zhong, Liangyou Li, Fei Mi, Lifeng Shang, Xin Jiang, Qun Liu, and Wei Wang. 2024. FollowBench: A Multi-level Fine-grained Constraints Following Benchmark for Large Language Models. arXiv:2310.20410. https://arxiv.org/abs/2310.20410

[26] Yuxuan Jiang, Meng Xu, Yiwen Song, and Xinwen Fu. 2025. AgentSight: Bridging the Semantic Gap Between Agent Intent and Low-Level Actions. arXiv:2508.02736. https://arxiv.org/abs/2508.02736

[27] Vasileios P. Kemerlis, Georgios Portokalidis, Kangkook Jee, and Angelos D. Keromytis. 2012. libdft: Practical Dynamic Data Flow Tracking for Commodity Systems. In *Proceedings of the 8th ACM SIGPLAN/SIGOPS Conference on Virtual Execution Environments*. Association for Computing Machinery, London, United Kingdom, 121–132. doi:10.1145/2151024.2151042

[28] Maxwell Krohn, Alexander Yip, Micah Brodsky, Natan Cliffer, M. Frans Kaashoek, Eddie Kohler, and Robert Morris. 2007. Information Flow Control for Standard OS Abstractions. In *Proceedings of the 21st ACM SIGOPS Symposium on Operating Systems Principles (SOSP ’07)*. ACM, 321–334.

[29] KubeArmor Authors. 2026. KubeArmor: Cloud Native Runtime Security Enforcement System. https://kubearmor.io/.

[30] Jiangming Li et al. 2025. ShieldAgent: Shielding Agents via Verifiable Safety Policy Reasoning. In *Proceedings of the International Conference on Machine Learning (ICML)*.

[31] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. In *Transactions of the Association for Computational Linguistics*, Vol. 12. 157–173.

[32] Jai Lal Lulla, Seyedmoein Mohsenimofidi, Matthias Galster, Jie M. Zhang, Sebastian Baltes, and Christoph Treude. 2026. On the Impact of AGENTS.md Files on the Efficiency of AI Coding Agents. arXiv:2601.20404. https://arxiv.org/abs/2601.20404

[33] Kiran-Kumar Muniswamy-Reddy, David A. Holland, Uri Braun, and Margo Seltzer. 2006. Provenance-Aware Storage Systems. In *2006 USENIX Annual Technical Conference (USENIX ATC 06)*. USENIX Association, Boston, MA, 43–56. https://www.usenix.org/conference/2006-usenix-annual-technical-conference/provenance-aware-storage-systems

[34] Andrew C. Myers. 1999. JFlow: Practical Mostly-Static Information Flow Control. In *Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL ’99)*. ACM, 228–241.

[35] OpenAI. 2025. Codex CLI. https://github.com/openai/codex.

[36] Thomas F. J.-M. Pasquier, Xueyuan Han, Thomas Moyer, Adam Bates, Olivier Hermant, David Eyers, Jean Bacon, and Margo Seltzer. 2018. Runtime Analysis of Whole-System Provenance. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. Association for Computing Machinery, Toronto, Canada, 1601–1616. doi:10.1145/3243734.3243776

[37] Thomas F. J.-M. Pasquier, Jatinder Singh, David Eyers, and Jean Bacon. 2017. CamFlow: Managed Data-Sharing for Cloud Services. *IEEE Transactions on Cloud Computing* 5, 3 (2017), 472–484. doi:10.1109/TCC.2015.2489211

[38] Fábio Perez and Ian Ribeiro. 2022. Ignore Previous Prompt: Attack Techniques For Language Models. arXiv:2211.09527. https://arxiv.org/abs/2211.09527

[39] Devin J. Pohly, Stephen McLaughlin, Patrick McDaniel, and Kevin Butler. 2012. Hi-Fi: Collecting High-Fidelity Whole-System Provenance. In *Proceedings of the 28th Annual Computer Security Applications Conference*. Association for Computing Machinery, Orlando, FL, 259–268. doi:10.1145/2420950.2420989

[40] Yunjia Qi, Hao Peng, Xiaozhi Wang, Amy Xin, Youfeng Liu, Bin Xu, Lei Hou, and Juanzi Li. 2025. AGENTIF: Benchmarking Instruction Following of Large Language Models in Agentic Scenarios. arXiv:2505.16944. https://arxiv.org/abs/2505.16944

[41] Traian Rebedea, Razvan Dinu, Makesh Sreedhar, Christopher Parisien, and Jonathan Cohen. 2023. NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, 431–445. https://aclanthology.org/2023.emnlp-demo.40

[42] Helio Victor F. Santos, Vitor Costa, Joao Eduardo Montandon, and Marco Tulio Valente. 2025. Decoding the Configuration of AI Coding Agents: Insights from Claude Code Projects. arXiv:2511.09268. https://arxiv.org/abs/2511.09268

[43] Edward J. Schwartz, Thanassis Avgerinos, and David Brumley. 2010. All You Ever Wanted to Know About Dynamic Taint Analysis and Forward Symbolic Execution (but Might Have Been Afraid to Ask). In *2010 IEEE Symposium on Security and Privacy*. IEEE, 317–331. doi:10.1109/SP.2010.26

[44] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. 2025. Progent: Programmable Privilege Control for LLM Agents. arXiv:2504.11703. https://arxiv.org/abs/2504.11703

[45] KP Singh. 2019. Kernel Runtime Security Instrumentation. Linux Plumbers Conference. https://lpc.events/event/4/contributions/454/

[46] The Linux Kernel Documentation. 2021. LSM BPF Programs. https://www.kernel.org/doc/html/v5.15/bpf/bpf_lsm.html.

[47] The Linux Kernel Documentation. 2025. Landlock: Unprivileged Access Control. https://www.kernel.org/doc/html/latest/userspace-api/landlock.html.

[48] Vivek Trivedy. 2026. The Anatomy of an Agent Harness. https://www.langchain.com/blog/the-anatomy-of-an-agent-harness.

[49] Haoyu Wang, Christopher M. Poskitt, and Jun Sun. 2026. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. 2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE). doi:10.1145/3744916.3764546

[50] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, et al. 2025. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. In *Proceedings of the 13th International Conference on Learning Representations*. https://openreview.net/forum?id=OJd3ayDDoF

[51] Robert N. M. Watson, Jonathan Anderson, Ben Laurie, and Kris Kennaway. 2010. Capsicum: Practical Capabilities for UNIX. In *19th USENIX Security Symposium (USENIX Security 10)*. USENIX Association, Washington, DC. https://www.usenix.org/conference/usenixsecurity10/capsicum-practical-capabilities-unix

[52] Zhen Xiang, Linzhi Zheng, Yanjie Li, Junyuan Hong, Qinbin Li, Han Xie, Jiawei Zhang, Zidi Xiong, Chulin Xie, Carl Yang, Dawn Song, and Bo Li. 2025. GuardAgent: Safeguard LLM Agents via Knowledge-Enabled Reasoning. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR. https://openreview.net/forum?id=2nBcjCZrrP

[53] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*. https://openreview.net/forum?id=mXpq6ut8J3

[54] Heng Yin, Dawn Song, Manuel Egele, Christopher Kruegel, and Engin Kirda. 2007. Panorama: Capturing System-wide Information Flow for Malware Detection and Analysis. In *Proceedings of the 14th ACM Conference on Computer and Communications Security*. Association for Computing Machinery, Alexandria, VA, 116–127. doi:10.1145/1315245.1315261

[55] Nickolai Zeldovich, Silas Boyd-Wickizer, Eddie Kohler, and David Mazières. 2006. Making Information Flow Explicit in HiStar. In *Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI '06)*. USENIX Association, 263–278.

[56] Jiongxiao Zhan, Yue Deng, Yiding He, Hongji Li, Jiaqi Li, Yibing Zhan, Wei Wang, Chaowei Xiao, and Bo Li. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated LLM Agents. In *Findings of the Association for Computational Linguistics: ACL 2024*.